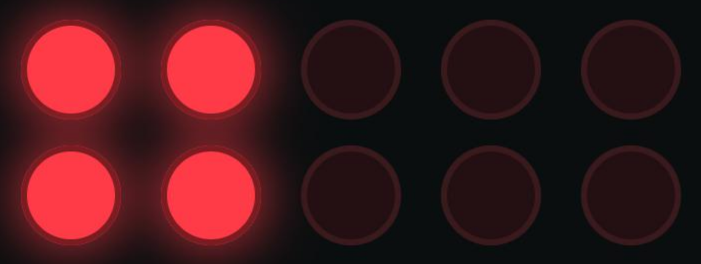
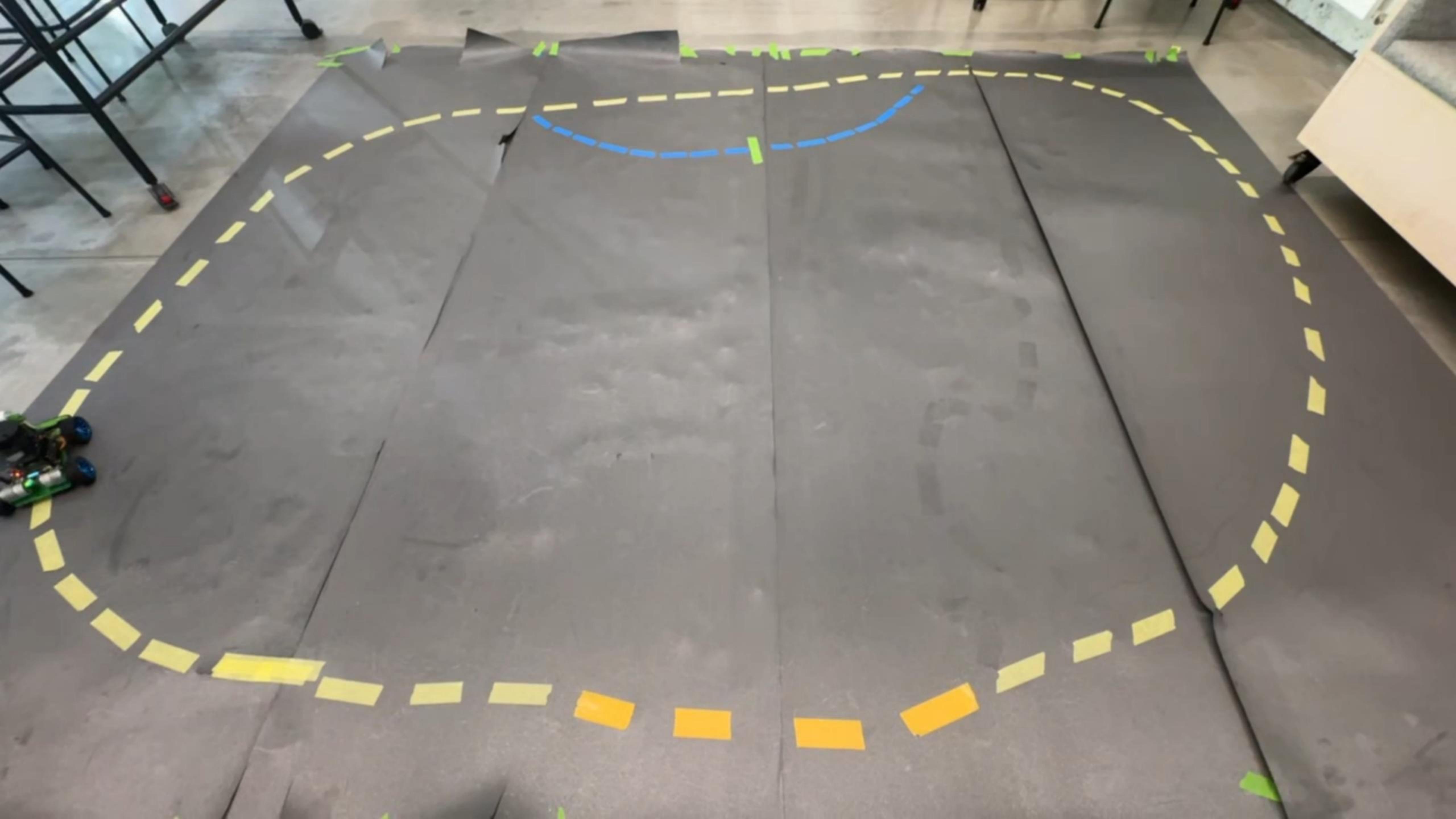


In racing, the fastest car does not win. The best-managed car wins.

Almost every small autonomous racer is built as if battery, tires and fuel were infinite. Ours is built the other way round: it watches all three run out, and plans around it.

Heineken
0.0
WHEN YOU DRIVE
NEVER DRINK
ROLEX
DHL
aramco
PIRELLI
Heineken
Heineken
0.0



BATTERY
41 %

// THE WHOLE TALK IN ONE PICTURE

Three resources. Three clocks. One race.

BATTERY

4.5
LAPS LEFT

Measured on the car. A real sensor reads the pack; the car projects how many laps that is worth.

TIRES

7.2
LAPS LEFT

Estimated, not measured. Wear is computed from how fast and how hard the car has been cornering.

■ SIMULATED · HAND-TUNED

FUEL

3.8
LAPS LEFT

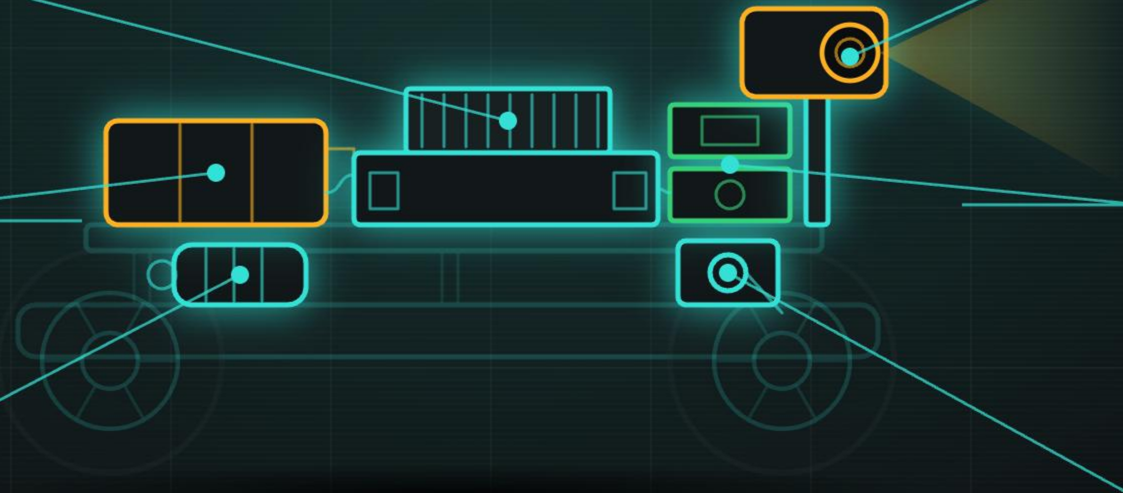
Estimated, not measured. Burned in proportion to throttle, exactly like a real combustion car.

■ SIMULATED · HAND-TUNED

Whichever clock hits zero first owns the race. Everything after this slide is those three clocks.

// ANATOMY · WHAT IS ACTUALLY BOLTED TO THE CAR

Six parts carry the whole cyber-physical loop



02
Jetson Nano
perception and physics, one board

04
2S LiPo pack
the resource being managed

05
Brushed motor
throttle becomes fuel burn

01
IMX219 camera
the only sensor that sees

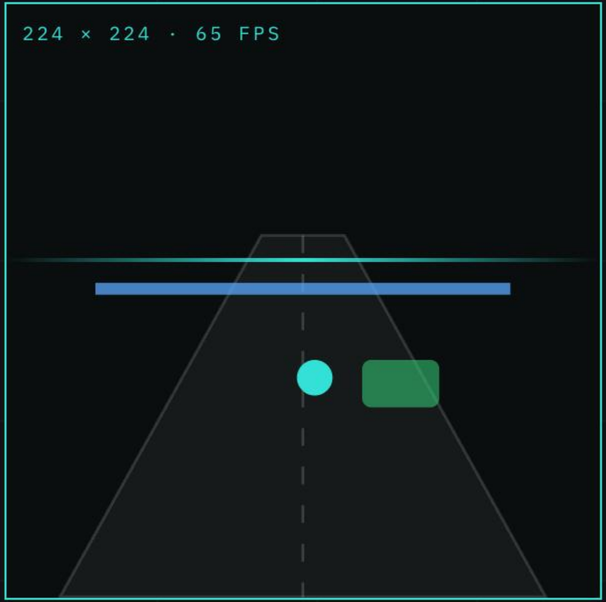
03
INA219 + TB6612
pack sensing, motor drive

06
Steering servo
tire wear caps this angle

CAMERA
65 FPS

// PERCEPTION · ONE CAMERA FRAME, THREE INDEPENDENT ANSWERS

What the car sees, and what it does about it



Where do I steer?

- A small neural network (ResNet18, TensorRT) returns **one point**: where on the track to aim.
- Trained by driving the track ourselves and labelling that point.

Is anyone in the way?

- YOLOv5 looks for people in the same frame.
- **2 frames in a row** → the car reverses. **5 clear frames** → all clear.
- It can override everything else.
- Tested over **40** person-in-path trials: **40/40** detected, **38/40** a full stop, **0** false positives.

`2 in a row → CONFIRMED · reverse`

Where am I on the track?

- Plain colour detection finds two painted markers.
- **green** = pit entry, held visible for **0.8 s**.
- **blue** = the lap line, re-armed once it leaves the frame.

BATTERY
41 %

01

ACT ONE

BATTERY

The only resource the car can actually measure.

// SMART BATTERY · RUNNING ON THE CAR

How many laps are left in the pack?

Every time the car crosses the blue lap line, it notes the battery level and compares it with the previous crossing. That difference is one lap's cost.

LAP N-4	8.2 %
LAP N-3	7.9 %
LAP N-2	8.4 %
LAP N-1	8.0 %
LAP N	7.5 %

Only the last **5** laps count. Older laps drop off the end.

Average cost of one lap	8.0 %
Battery in the pack right now	41 %
41 % ÷ 8.0 % per lap	5.1 laps
Keep a 5 % reserve so it never dies on track	−0.6
Laps we are actually allowed to run	4.5 laps

Step 1 · measure → average → project → subtract the margin

DECISION
BOX

// THE DECISION, EXACTLY AS THE CAR MAKES IT

Four laps to the finish. Do we make it?

Battery in the pack	35 %
4 laps still to run, at 8 % each	32 %
plus the 5 % safety reserve	37 %

35 % < 37 % **BOX**

- We do not make it. The car calls itself in.
- Takes on charge, rejoins once the pack is back to **70 %**.
- No human touches anything.

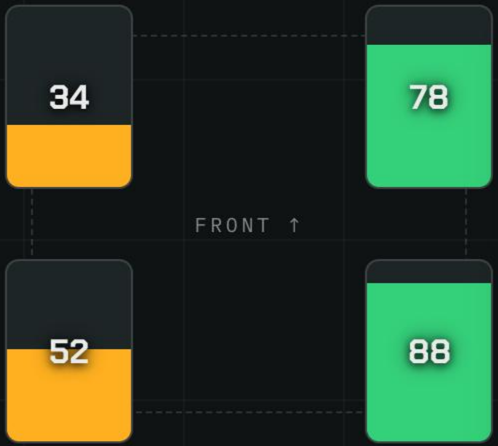
ACT TWO

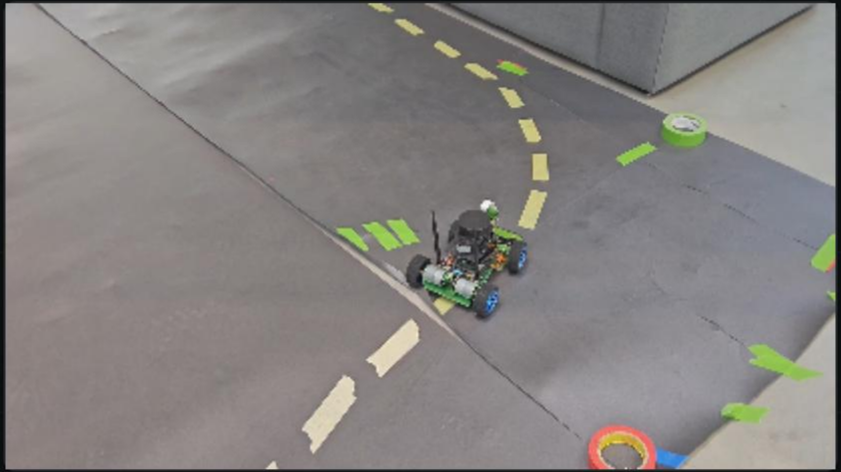
TIRES

The resource we can only infer.

// VIRTUAL SENSOR · TIRE WEAR

No tire sensor exists at this scale. So we compute it.





Wear is charged for cornering, not for driving

- Going fast in a straight line costs almost nothing.
- Going fast while turning costs a lot.
- The car adds up speed × steering angle over time and calls that wear.

Each corner wears differently

- The outside front tire takes the most load, so it goes first. Exactly like a real car.
- All four are tracked and streamed separately.

A floor at 25 % forces the stop

- Below 25 % remaining, the tires are declared finished.
- A pit stop becomes mandatory, whatever the battery says.

wear rate ≈ $k \times \text{speed}^2 \times \text{steering}$ · k hand-tuned from test laps

// THE CYBER-PHYSICAL MOMENT

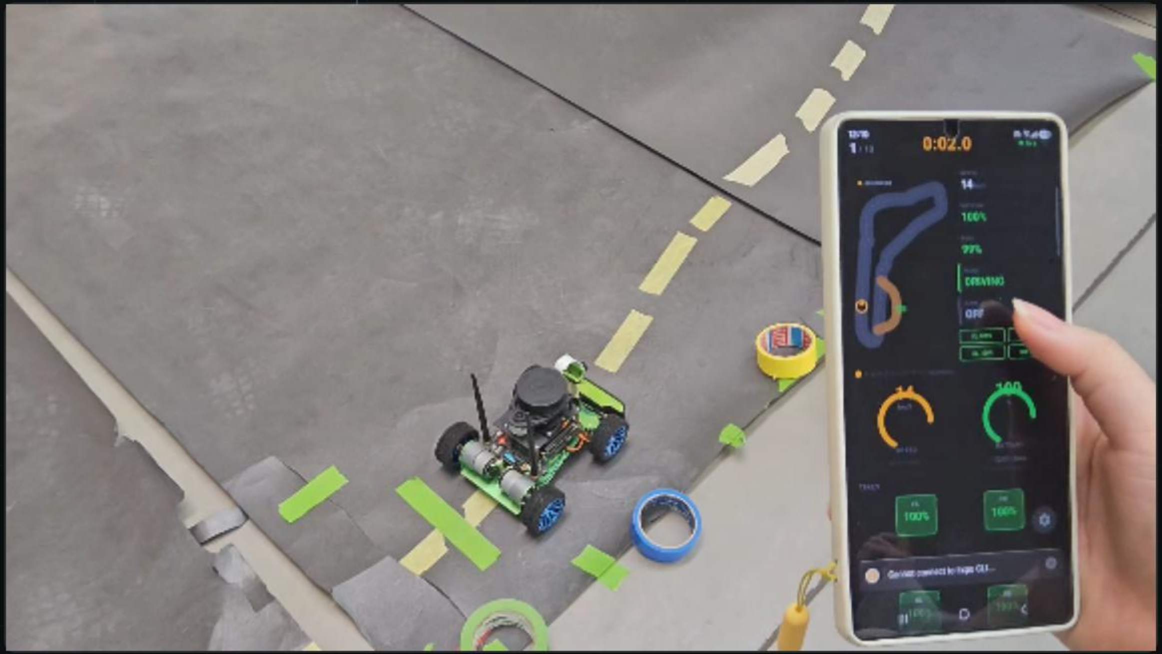
A phone changes how fast the car is allowed to go

```
Someone taps “Rain” in the app:
race/car1/command/rain
{ "command": "rain", "enabled": true }
```

- The message goes phone → broker → car.
- The car handles it on a **separate listener**, so the driving loop never pauses to read mail. It just sees a flag flip on the next frame.

MAXIMUM THROTTLE 0.07

Less grip available: the car caps itself at roughly a third of its dry speed. Nothing else in the code changed.



ACT THREE

FUEL

The resource that makes the car faster as it disappears.

// VIRTUAL SENSOR · FUEL

■ SIMULATED · HAND-TUNED · NOT BENCH-CALIBRATED

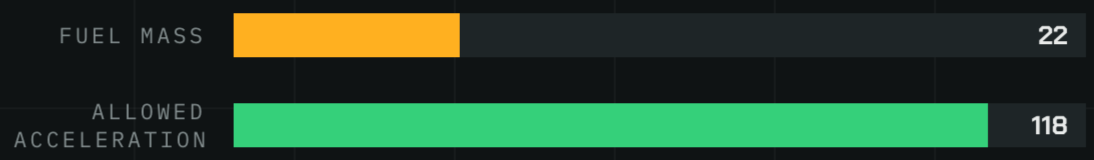
FUEL · LAPS
3.8

The clock that runs backwards

- The car burns fuel in proportion to throttle.
- Full throttle costs about **0.15 % per second**, half throttle about half that.
- Below **15 %** remaining, a stop is mandatory.

```
burn rate ≈ k × throttle  ·  streamed on race/car1/fuel
```

- This is the one resource whose disappearance is **partly good news**.
- An emptier car is a lighter car, and a lighter car may accelerate harder.
- Real racing works exactly this way.



One goes down, the other goes up, in the same moment.

STATE
PIT SERVICE

One car, three deadlines, one pit lane

BATTERY 0.0

TIRES 2.1

FUEL 3.8

Battery is out of laps. Tires are within two. **Box now, do both.**

The planner does the simplest sensible thing: it **obeys the smallest clock**, the lap on which the car has to be in the pits.

If a second clock runs out within **two laps** of the first, it is served in the **same stop**. Stopping twice costs more time than it saves.

RACING
DRIVE THE LINE

→

PIT ENTRY & SERVICE
GREEN HELD 0.8 S · ~10 S STOP

→

PIT EXIT
REJOIN AT 70 %

Seeing the green marker is not enough on its own. The car turns in only if it has already decided to box.

SAFETY OVERRIDE Person detected → stop and reverse. Sits on top of all three states and cannot be outvoted.

TELEMETRY
~80 ms

// TELEMETRY · GETTING THE NUMBERS OFF THE CAR

The car races. The phone watches.

`race/car1/state` `race/car1/battery` `race/car1/fuel`
`race/car1/tires` `race/car1/imu` `race/car1/command/rain`

Capture the frame	15 ms
Run the network	25 ms
Decide	5 ms
Publish → broker → app	35 ms

~80 ms against a 100 ms target, fast enough for a human on the pit wall to react.

■ ESTIMATE · PER STAGE, NOT END-TO-END

JetRacer
publishes over plain MQTT port 1883

telemetry →

Mosquitto broker
on a local Wi-Fi hotspot the only thing both sides know about

← commands

Expo phone app
MQTT over WebSocket port 9001

One glance, and you know everything the car knows

- This is the live dashboard during a real pit stop, running on a phone over the same broker.
- Mode reads **CHARGING**. The battery climbs in real time, from the twenties back toward seventy.
- Every tile, speed, battery, fuel, all four tires, the mode, is a message the car published a fraction of a second ago.
- The phone never commands the car to charge. The car decided that itself; the app just shows you it happening.

`race/car1/state · race/car1/battery · the pit wall, in your pocket`

// FULL SCENE · ONE LAP WITH EVERYTHING SWITCHED ON

Watch the three clocks fight each other

TRACK CONDITION

DRY · grip 1.00

Rain is triggered from the app, not sensed.

LAP 01	STATE RACING
BATTERY 98%	TIRES 98%
FUEL 99%	GRIP CAP 1.00µ

Rain arrives, grip falls, the tires give up early. The planner folds the tire stop into the battery stop: one visit, both jobs.

BATTERY
41 %

// FROM HERE TO A FINISHED LOOP

What's built, and where this grows next

✓ **Running on the real car**

- Camera-based steering — 0.042 test MAE, 32 FPS on TensorRT (2.1× over PyTorch)
- Person detection with reverse — 40/40 detected, 38/40 full stop, 0 false positives
- Green pit marker and blue lap line
- 5-lap battery projection with its 5 % reserve
- Racing ⇔ pit-stop state machine — 10/12 laps completed on the test track
- Rain mode and live telemetry to the phone

→ **Specified, not yet closed into the loop**

- Tire and fuel estimates still live on the simulation side
- The full three-way planner
- Bench calibration of the two hand-tuned constants
- A real humidity sensor instead of an app button

The fastest lap time is not achieved by ignoring the health of the machine, but by managing it.

Kelly · Fiona · Vincent · Soyeong
Gang-Yun · Angel · Katrin

EMBEDDED SYSTEMS, CYBER-PHYSICAL SYSTEMS AND ROBOTICS
TUM SCHOOL OF CIT · PROF. DR. AMR ALANWAR
9 SEPTEMBER 2026